Leetcode 23. Merge k Sorted Lists

September 25th 2026

Understanding the Problem

In this problem, we are given k linked lists that are each sorted in ascending order. Our goal is to return one linked list that contains all the elements from each of the k lists, sorted in ascending order.

Brute Force Approaches

There are many ways to solve this problem, including some brute force methods. Note that in the time and space complexities, ℓ is the length of the **longest** list among the k lists.

Merging 2 at a Time

This strategy involves repeating the two-pointer algorithm used to solve Merge Two Sorted Lists, applying the algorithm to every list. This can be implemented by adding all lists to a stack, popping the first 2 lists, combining them, and pushing the combined list back into the stack.

- Time complexity: $\mathcal{O}(k \sum_{\ell=1}^{k} \ell_i)$
- Space complexity: $\mathcal{O}(k)$

Each merge causes the amount of work to keep adding up until the final merge needs to be made.

Put all Nodes in a List

Simply create a `List` storing all nodes from all the lists from the input. Next, sort and then iterate through the `List` and connect the nodes. For this to work, ensure that the nodes are sorted by their value (`node.val`).

- Time complexity: $\mathcal{O}(\ell k \log(\ell k))$
- Space complexity: $\mathcal{O}(\ell k)$

Since we have to sort the combined list, the time complexity will be proportional to $n \log(n)$, where n is the total number of nodes from all lists.

Efficient Solution

This more efficient solution involves using ideas from both the brute force solutions discussed above. First, it's important to efficiently retrieve the smallest-valued node at each step. To do this, we can use a `PriorityQueue`(min-heap) of nodes. Here is the algorithm:

1. Put the head of every list in a `PriorityQueue`
2. Create a dummy node (`ans`) and a copy of the dummy node (`head`) to connect the nodes.
3. While the queue is not empty:
    - Get the smallest-valued node from the queue (`curr=pq.poll()`)
    - Add `curr.next` into the queue to signal that `curr` has been processed, if `curr != null`.
4. Return the final combined list (`ans.next`)

Complexity Analysis

- Time complexity: $\mathcal{O}(\ell k \log k)$
- Space complexity: $\mathcal{O}(k)$

Placing each node in the final list takes $\log k$ time since there will be at most k lists in the queue at one time. Since k lists need to be stored in the queue, the space complexity is $\mathcal{O}(k)$.